

» Derivative-Free Methods

- * Useful when derivative isn't available, and some derivative-free methods (such as annealing, global search) also can help when function has many local minima
- * Some applications in ML: *hyperparameter optimisation*, *feature selection* e.g. see
 - * *Keras* https://keras.io/keras_tuner/
 - * *Facebook Nevergrad*
<https://engineering.fb.com/2018/12/20/ai-research/nevergrad/>,
<https://github.com/facebookresearch/nevergrad>
 - * *scikit-optimize* <https://scikit-optimize.github.io/stable/>, *HyperOpt*
<https://hyperopt.github.io/hyperopt/>, *Dlib* <http://blog.dlib.net/2017/12/a-global-optimization-algorithm-worth.html>,
RayTune <https://docs.ray.io/en/latest/tune.html>
- * For an overview of derivative-free methods see e.g.
 - * *Derivative-free optimization: a review of algorithms and comparison of software implementations, 2013.*
<https://link.springer.com/article/10.1007%2Fs10898-012-9951-y>
 - * *Derivative-free optimization methods, 2019.*
<https://arxiv.org/pdf/1904.11585.pdf>
 - * For python this comparison site is worth a look:
<https://www.microprediction.com/blog/humpday>

» Derivative-Free Methods

Here we'll just try to summarise some of the main ideas commonly used:

- * *Approximate gradients* - we talked about them in last lecture
 - * e.g. finite difference approx, Nesterov random search
- * *Local search* - directly search for nearby points where cost function is lower, similar in spirit to gradient descent but doesn't use gradient
 - * e.g. pure random search, pattern search, simplex method/Nelder-Mead, CMA-ES
- * *Local trust-region/model-based methods* - locally approximate cost function, e.g. using quadratic, then minimise but restrict any move to be near to starting point ("trust region")
- * *Global search*
 - * e.g. grid search, global random search, hit and run, simulated annealing
- * *Global model-based methods* - sample points from cost function, use these to build an approx model (e.g. by smoothly interpolating sampled points)
 - * e.g. bayesian optimisation

» Local Search: Pure Random Search

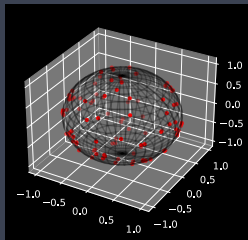
- * Really simple: randomly pick a new point, if its better then move to it, otherwise stay still. Repeat. Pseudo code:

```
→ x=x0
  for k in range(num_iters):
    → select a random point y
    if  $f(y) < f(x)$ :
      →  $x = y$  ✓
```

- * Obvious extension: try several new points, pick the best one (the one for which function f is smallest) as $y \rightarrow$ will come back to this shortly

» Local Search: Pure Random Search

- * New points $y = x + \delta$
- * Draw δ uniformly at random from the surface of the unit sphere¹
- * Generate a vector z with each element Gaussian distributed. Set $\delta = z / \sum_{i=1}^n z_i^2$. In python:
 - `z = np.random.randn(ndim, npoints)`
`delta = z / np.sqrt(np.sum(z*z,axis=0))`
 - `y = x + delta`

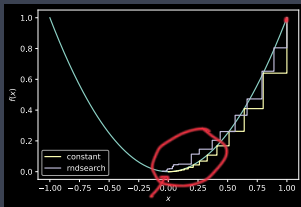


- * The radius of the sphere is our step-size. We can use a constant value, or adapt it over time e.g. decrease the radius when we see the cost function stop decreasing

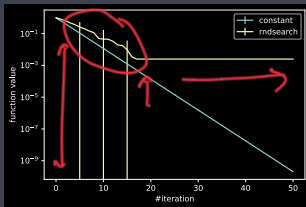
¹See https://en.wikipedia.org/wiki/N-sphere#Generating_random_points

» Examples

- * $f = x^2$, starting point $x = 1$, grad descent with constant stepsize
 $\alpha = 0.1$, random search $\delta = 0.21$



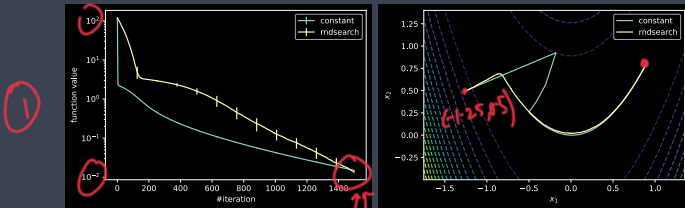
rnd search data is mean and std dev over 25 runs



- * Random search initially converges quite quickly, but then stops → when close to the minimum smaller steps are needed, otherwise just bounce around searching in vicinity of minimum but not finding a point that decreases $f(\cdot)$.

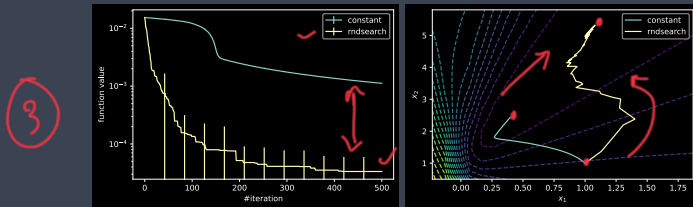
» Examples

- * Rosenbrock function: grad descent with constant stepsize $\alpha = 0.002$, random search $\delta = 0.01$



rnd search data is mean and std dev over 25 runs

- * Toy neural net loss: grad descent with constant stepsize $\alpha = 0.75$, random search $\delta = 1.5$

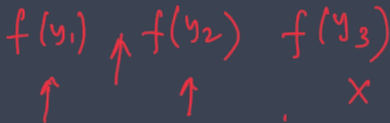


rnd search data is mean and std dev over 25 runs

» Pure Random Search: Adapting The Step Size

In above examples we used a constant step size. As get close to minimum this can stop search from converging. Some ideas for adapting the step size include:

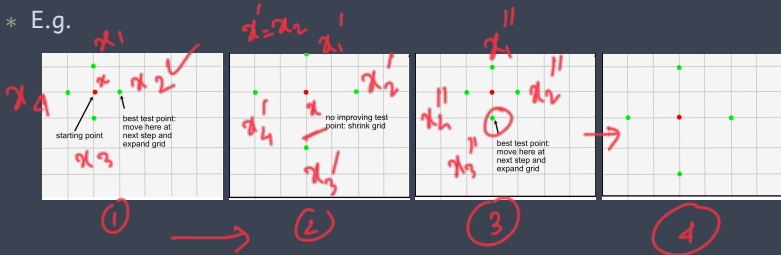
- * Start with step size α . When new random point y fails to decrease $f(\cdot)$, decrease α e.g. $\alpha = 0.95\alpha$
- * Pick a step size α uniformly at random between 0 and a maximum value → "hit and run" $[0, 0.95]$
- * Pick three points $y_1 = x + \gamma_1\alpha\delta$, $y_2 = x + \alpha\delta$, $y_3 = x + \gamma_2\alpha\delta$ with $\gamma_1 < 1$ and $\gamma_2 > 1$. Pick the point which makes $f(\cdot)$ smallest and update α to be the step size of that point, e.g. if point is y_1 then $\alpha = \gamma_1\alpha$.
- * *Using multiple points to explore more quickly and adjust the step size is v common e.g. pattern search, simplex method*



$f(y)$
 $f(x)$

→ » Local Search: Pattern Search²

- * Maintain grid of test points around current point x
- * If cost function is lower at one of test points, set $x = \text{test point}$ and expand grid e.g. by $\times 2$
- * If no test point as lower cost function, shrink grid e.g. by $\times 0.5$
- * E.g.

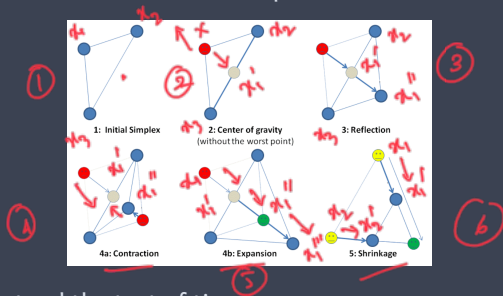


- * Simple, but often works surprisingly well ↑

²E.g. see <https://www.gerad.ca/Charles.Audet/PUB/aad2.pdf>

» Local Search: Simplex Method/Nelder-Mead Method³

- * Vector x has n elements, simplex method $\rightarrow$ maintain $n + 1$ test points
- * Based on test point with worst function value adjust the set of test points by: reflection, contraction, expansion:

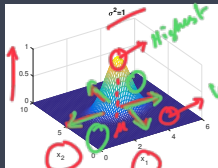


- * Robust, has stood the test of time

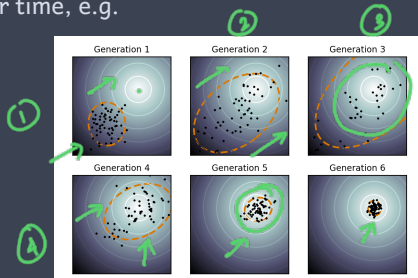
³E.g. see https://en.wikipedia.org/wiki/Nelder%E2%80%93Mead_method

» Local Search: CMA-ES⁴

- * Test points are randomly sampled from a Gaussian distribution centered on current point.



- * Covariance (spread of values and ellipsoidal shape) of distribution is adjusted over time, e.g.



source: <https://en.wikipedia.org/wiki/CMA-ES>

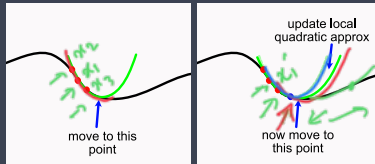
- * Often competitive

⁴E.g. see <https://en.wikipedia.org/wiki/CMA-ES>

» Local Trust Region Methods

- * Fit a local quadratic approximation/model to function around current point x (use function value at previous values of x to help with this)
- * Move to minimum of quadratic approx
- * Repeat

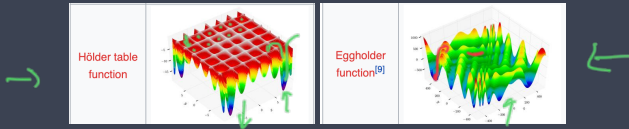
$$-f(x_3) < -f(x_1)$$



- * *Trust region*: restrict the size of the move made. Adapt size of trust region based on progress e.g.
 - * If when move to next point the cost function decrease is similar to prediction from quadratic model then increase trust region
 - * If cost function is v different from expected (e.g. function increases rather than decreases) then decrease the trust region
- * Not restricted to use of a local quadratic approx, can use other models too e.g. kernel method
- * Often used as last step in an optimisation e.g. use random search to get to the vicinity of minimum, then use trust region approach to converge more quickly and accurately onto minimum

» Global Methods

- * So far we've looked at local search methods → they all try to move downhill by taking small-ish steps
- * But suppose we have a cost function like this:

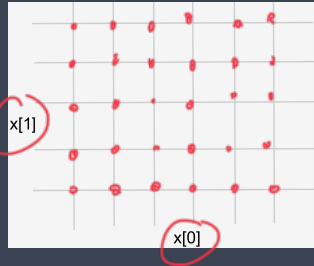


- * Local search is unlikely to work well - will tend to get stuck in a local minimum, which might be quite poor
- * Global methods target this kind of task. Two main approaches:
 - * *Global search* e.g. grid search, global random search, hit and run, simulated annealing
 - * *Global model-based methods* - sample points from cost function, use these to build an approx model (e.g. by smoothly interpolating sampled points) e.g. bayesian optimisation

» Global Search: Grid Search

- * “Brute-force” grid search:
- * Suppose x has two elements:

```
for x0 in np.arange(-1,1,0.1):  
    for x1 in np.arange(-1,1,0.1):  
        x=[x0,x1]  
        evaluate f(x)  
        if smallest value so far:  
            keep a note of x, f(x)
```

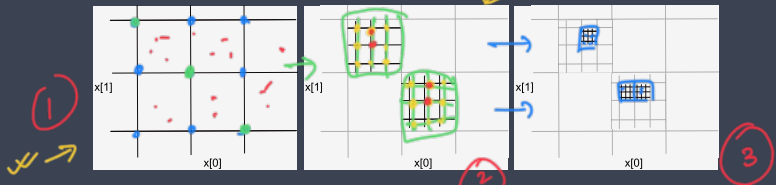


- * E.g. `GridSearchCV()` in sklearn
- * Simple, but expensive (many function evaluations) and doesn't scale to x with more than a few elements

» Global Search: Branch and Bound Methods

* “Smarter” grid search:

- * Start off with a coarse grid
- * Evaluate a test point (or maybe a few) in each grid square.
- * Prune grid square squares where function value is poor
- * For remaining grid squares now use a finer grid, and repeat test/prune/refine process



- * E.g. *Dividing RECTangle (DIRECT)* algorithm, 1993

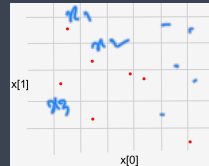
<https://link.springer.com/article/10.1007/BF00941892> → has stood the test of time, *Multilevel coordinate search (MCS)*

https://en.wikipedia.org/wiki/MCS_algorithm

» Global Search: Random Search

- * Suppose $x = [x_0, x_1]$ has two elements. Define an allowed range of values for each element e.g. $x_0\text{range} = [-1, 1]$, $x_1\text{range} = [0, 10]$.

```
for k in range(num_iters):  
    pick x0 uniformly at random  
    pick x1 uniformly at random  
    x=[x0,x1]  
    evaluate f(x)  
    if smallest value so far:  
        keep a note of x, f(x)
```



- * And similarly when x has more elements.
- * Popularised for hyperparameter search by this paper: Random Search for Hyper-Parameter Optimization, 2012
<https://www.jmlr.org/papers/volume13/bergstra12a/bergstra12a.pdf>.
- * Available in Keras, PyTorch/RayTune, sklearn

» Global Search: Population Based Search

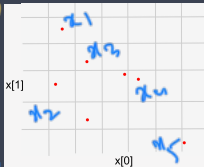
- * A variant on random search proposed by DeepMind for hyperparameter tuning, see *Population Based Training of Neural Networks 2017* <https://arxiv.org/pdf/1711.09846.pdf>
 - * Start with a randomly chosen set of N test points $\{x_1, \dots, x_N\}$, evaluate cost function at these points.
 - * Prune poorly performing test points i.e. points where cost function is worst. Take the best points, perturb them randomly, replace pruned points with these

1

$$f(x_3) = \min \{f(x_i) \mid i=1,5\}$$

→

$$= \min \{f(x_1), f(x_2), \dots, f(x_5)\}$$



- * Includes early stopping - no need to complete full training once notice that a choice of hyperparameters is poor
- * Available in PyTorch/RayTune <https://docs.ray.io/en/latest/tune.html>
- * *Hyperband* uses a broadly similar approach, see *Hyperband: A Novel Bandit-Based Approach to Hyperparameter Optimization 2018* <https://jmlr.csail.mit.edu/papers/v18/16-558.html>. Available in PyTorch/RayTune, Keras, sklearn.